U4U Engine: Personalized Genomic Analysis Platform

Document Type: Technical Flowchart — Innovation / Patent Disclosure

Prepared for: UF Innovate — University of Florida Technology Transfer Office

Date: June 16, 2026

Inventors: Noah Thomas Jones, Curtis Dearing

Technology Summary: A multi-layer software system that ingests raw genomic data (VCF, 23andMe, CSV, or rsID lists), performs automated parallel variant annotation against seven curated external biomedical databases, applies ACMG/AMP evidence-based pathogenicity classification, generates pharmacogenomic star-allele phenotype predictions with Mondrian conformal confidence sets, estimates ancestry-adjusted polygenic risk scores, maps peptide therapy candidates to discovered gene variants, and surfaces curated plus live FDA regulatory status — all via a REST API with an asynchronous job queue, Fernet-encrypted job persistence, and longitudinal Bayesian biomarker tracking.

Diagram 1 — System Architecture Overview

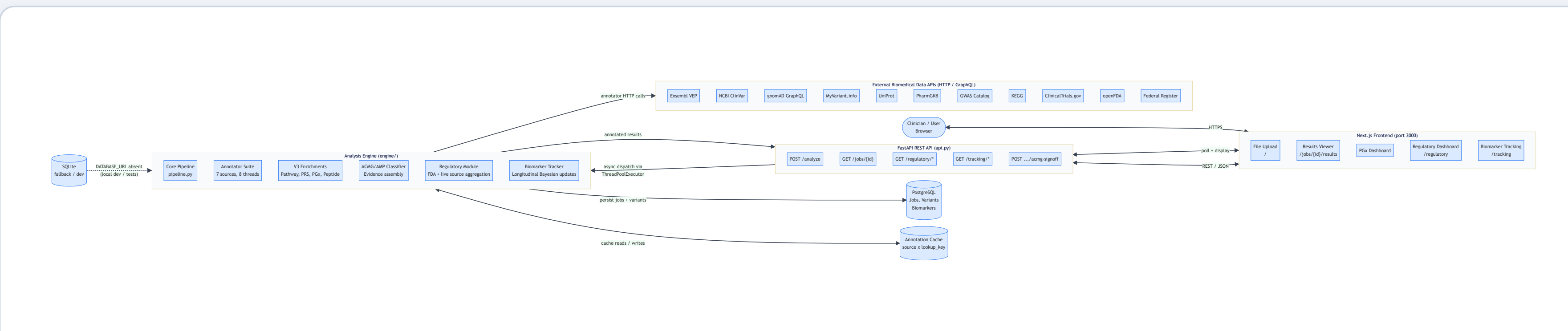
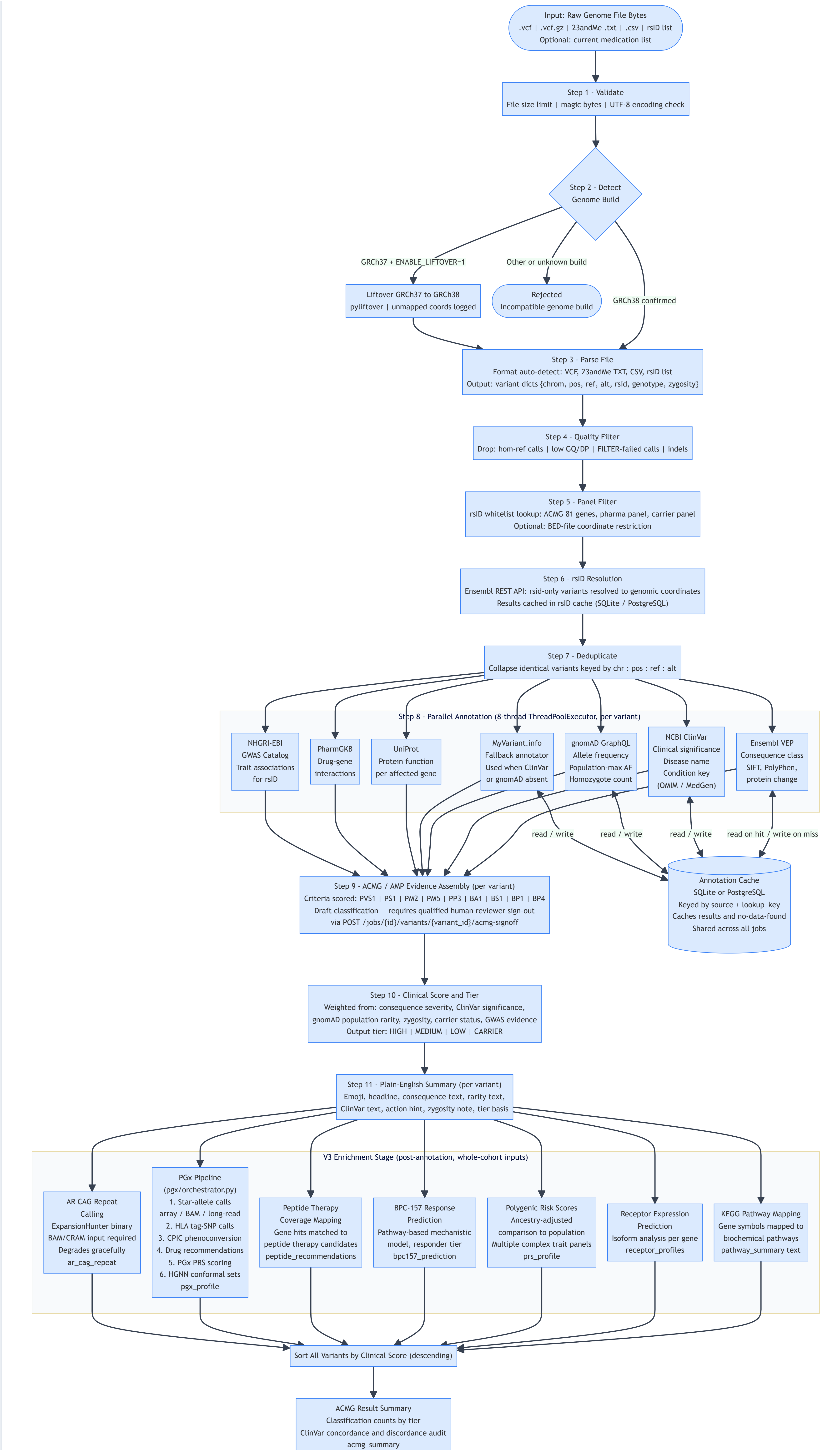


Figure 1. High-level system architecture. Users upload genome files through the Next.js browser frontend. The FastAPI backend enqueues each job and responds immediately with a job ID (HTTP 202); the client polls until analysis completes. The analysis engine runs in a thread pool to keep the async event loop free during long-running external API calls. All annotation calls are backed by a shared cache (SQLite in development, PostgreSQL in production). Completed results are Fernet-encrypted and persisted to PostgreSQL.

Diagram 2 — Core Analysis Pipeline Detail (engine/pipeline.py)


Input: Raw Genome File Bytes
.vcf | .vcf.gz | 23andMe .txt | .csv | rsID list
Optional: current medication list
Step 1 - Validate
File size limit | magic bytes | UTF-8 encoding check
Step 2 - Detect Genome Build
GRCh37 + ENABLE_LIFTOVER=1
Other or unknown build
GRCh38 confirmed
Liftover GRCh37 to GRCh38
pyliftover | unmapped coords logged
Rejected
Incompatible genome build
Step 3 - Parse File
Format auto-detect: VCF, 23andMe TXT, CSV, rsID list
Output: variant dicts {chrom, pos, ref, alt, rsid, genotype, zygosity}
Step 4 - Quality Filter
Drop: hom-ref calls | low GQ/DP | FILTER-failed calls | indels
Step 5 - Panel Filter
rsID whitelist lookup: ACMG 81 genes, pharma panel, carrier panel
Optional: BED-file coordinate restriction
Step 6 - rsID Resolution
Ensembl REST API: rsid-only variants resolved to genomic coordinates
Results cached in rsID cache (SQLite / PostgreSQL)
Step 7 - Deduplicate
Collapse identical variants keyed by chr : pos : ref : alt
Step 8 - Parallel Annotation (8-thread ThreadPoolExecutor, per variant)
NHGRI-EBI GWAS Catalog
Trait associations for rsID
PharmGKB
Drug-gene interactions
UniProt
Protein function per affected gene
MyVariant.info
Fallback annotator
Used when ClinVar or gnomAD absent
gnomAD GraphQL
Allele frequency
Population-max AF
Homozygote count
NCBI ClinVar
Clinical significance
Disease name
Condition key
(OMIM / MedGen)
Ensembl VEP
Consequence class
SIFT, PolyPhen, protein change
read / write
read / write
read / write
read on hit / write on miss
Annotation Cache
SQLite or PostgreSQL
Keyed by source + lookup_key
Caches results and no-data-found
Shared across all jobs
Step 9 - ACMG / AMP Evidence Assembly (per variant)
Criteria scored: PVS1 | PS1 | PM2 | PM5 | PP3 | BA1 | BS1 | BP1 | BP4
Draft classification — requires qualified human reviewer sign-out
via POST /jobs/{id}/variants/{variant_id}/acmg-signoff
Step 10 - Clinical Score and Tier
Weighted from: consequence severity, ClinVar significance,
gnomAD population rarity, zygosity, carrier status, GWAS evidence
Output tier: HIGH | MEDIUM | LOW | CARRIER
Step 11 - Plain-English Summary (per variant)
Emoji, headline, consequence text, rarity text,
ClinVar text, action hint, zygosity note, tier basis
V3 Enrichment Stage (post-annotation, whole-cohort inputs)
AR CAG Repeat Calling
ExpansionHunter binary
BAM/CRAM input required
Degrades gracefully
ar_cag_repeat
PGx Pipeline
(pgx/orchestrator.py)
1. Star-allele calls
array / BAM / long-read
2. HLA tag-SNP calls
3. CPIC phenoconversion
4. Drug recommendations
5. PGx PRS scoring
6. HGNN conformal sets
pgx_profile
Peptide Therapy Coverage Mapping
Gene hits matched to peptide therapy candidates
peptide_recommendations
BPC-157 Response Prediction
Pathway-based mechanistic model, responder tier
bpc157_prediction
Polygenic Risk Scores
Ancestry-adjusted comparison to population
Multiple complex trait panels
prs_profile
Receptor Expression Prediction
Isoform analysis per gene
receptor_profiles
KEGG Pathway Mapping
Gene symbols mapped to biochemical pathways
pathway_summary text
Sort All Variants by Clinical Score (descending)
ACMG Result Summary
Classification counts by tier
ClinVar concordance and discordance audit
acmg_summary

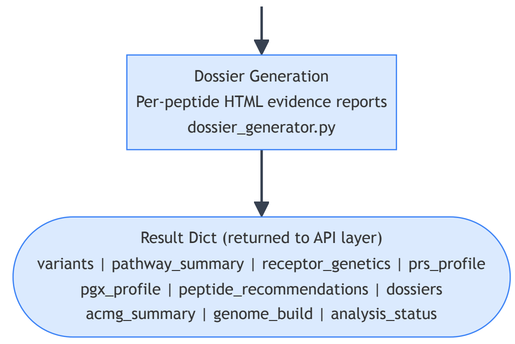


Figure 2. Detailed core analysis pipeline (engine/pipeline.py). Raw genome file bytes enter at top and pass through eleven sequential processing steps. Step 8 (annotation) runs in an 8-thread pool — each variant is independently annotated against Ensembl VEP, NCBI ClinVar, gnomAD, MyVariant.info (fallback), UniProt, PharmGKB, and the GWAS Catalog, with all results backed by a shared cache. The V3 enrichment stage runs after the per-variant loop and operates on the full variant cohort to produce pathway, receptor, PRS, PGx, BPC-157, and peptide outputs. The ACMG/AMP classification in Step 9 is an automated evidence-assembly aid and must not serve as a final clinical determination without a qualified reviewer's countersignature via the dedicated API endpoint.

Diagram 3 — Pharmacogenomics Sub-Pipeline (engine/pgx/)

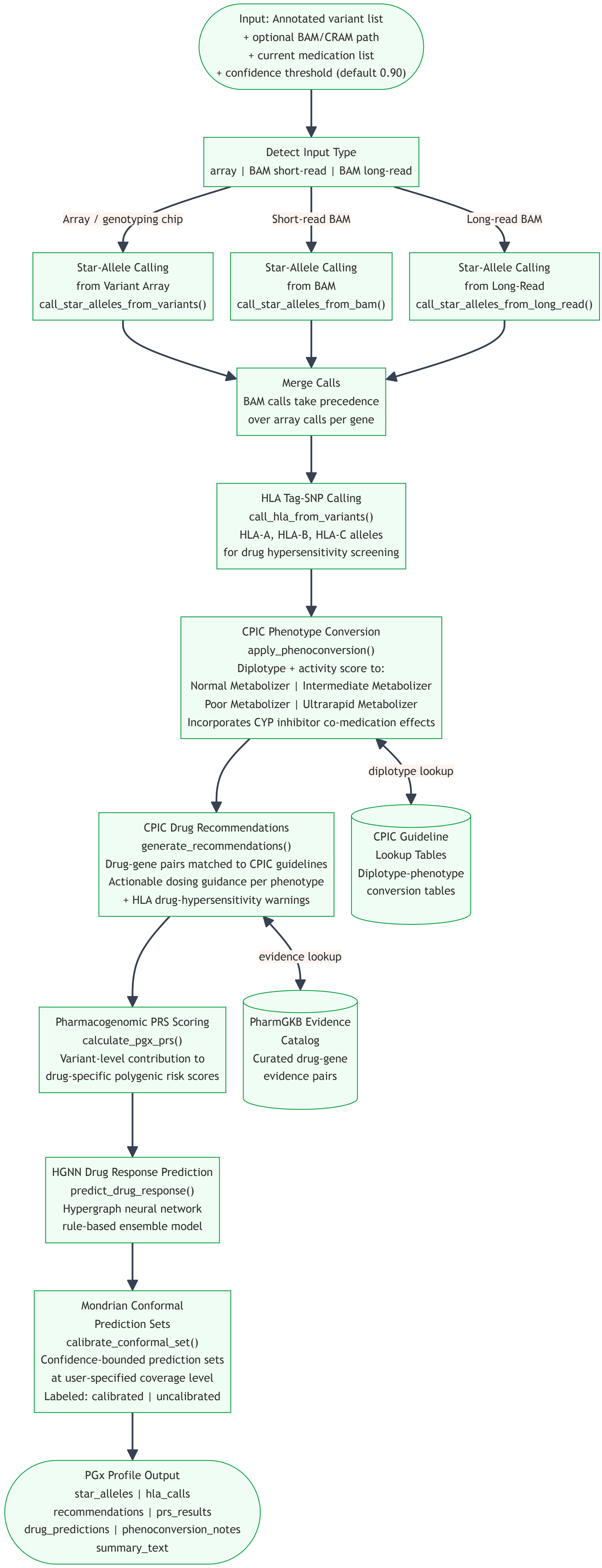

Input: Annotated variant list
+ optional BAM/CRAM path
+ current medication list
+ confidence threshold (default 0.90)
Detect Input Type
array | BAM short-read | BAM long-read
Array / genotyping chip
Short-read BAM
Long-read BAM
Star-Allele Calling
from Variant Array
call_star_alleles_from_variants()
Star-Allele Calling
from BAM
call_star_alleles_from_bam()
Star-Allele Calling
from Long-Read
call_star_alleles_from_long_read()
Merge Calls
BAM calls take precedence
over array calls per gene
HLA Tag-SNP Calling
call_hla_from_variants()
HLA-A, HLA-B, HLA-C alleles
for drug hypersensitivity screening
CPIC Phenotype Conversion
apply_phenoconversion()
Diplotype + activity score to:
Normal Metabolizer | Intermediate Metabolizer
Poor Metabolizer | Ultrarapid Metabolizer
Incorporates CYP inhibitor co-medication effects
diplotype lookup
CPIC Guideline
Lookup Tables
Diplotype-phenotype
conversion tables
CPIC Drug Recommendations
generate_recommendations()
Drug-gene pairs matched to CPIC guidelines
Actionable dosing guidance per phenotype
+ HLA drug-hypersensitivity warnings
evidence lookup
PharmGKB Evidence
Catalog
Curated drug-gene
evidence pairs
Pharmacogenomic PRS Scoring
calculate_pgx_prs()
Variant-level contribution to
drug-specific polygenic risk scores
HGNN Drug Response Prediction
predict_drug_response()
Hypergraph neural network
rule-based ensemble model
Mondrian Conformal
Prediction Sets
calibrate_conformal_set()
Confidence-bounded prediction sets
at user-specified coverage level
Labeled: calibrated | uncalibrated
PGx Profile Output
star_alleles | hla_calls
recommendations | prs_results
drug_predictions | phenoconversion_notes
summary_text

Figure 3. Pharmacogenomics (PGx) sub-pipeline (engine/pgx/orchestrator.py). Star-allele calls are derived from array genotypes, short-read BAM, or long-read BAM (BAM calls override array calls for the same gene). HLA tag-SNPs are called independently to screen for drug hypersensitivity risk. CPIC phenoconversion translates star-allele diplotypes into metabolizer phenotypes, accounting for co-medication CYP inhibitor effects. A hypergraph neural network (HGNN) ensemble produces drug-response predictions bounded by Mondrian conformal prediction sets at a user-specified confidence level. Without a validated calibration set, predictions are labeled "uncalibrated."

Diagram 4 — Regulatory Dashboard Module (engine/regulatory/)

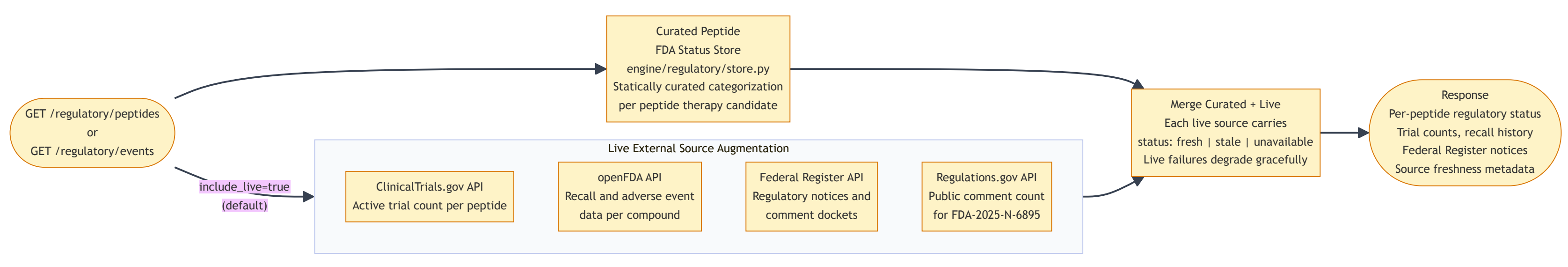


Figure 4. Regulatory dashboard module (engine/regulatory/). A curated static store of FDA regulatory status for each peptide therapy candidate is merged at request time with live data from four external government APIs. Each live source independently reports its own freshness status; failures in any live source degrade gracefully and do not fail the request.

Diagram 5 — Bayesian Biomarker Inference Model (engine/tracking/)

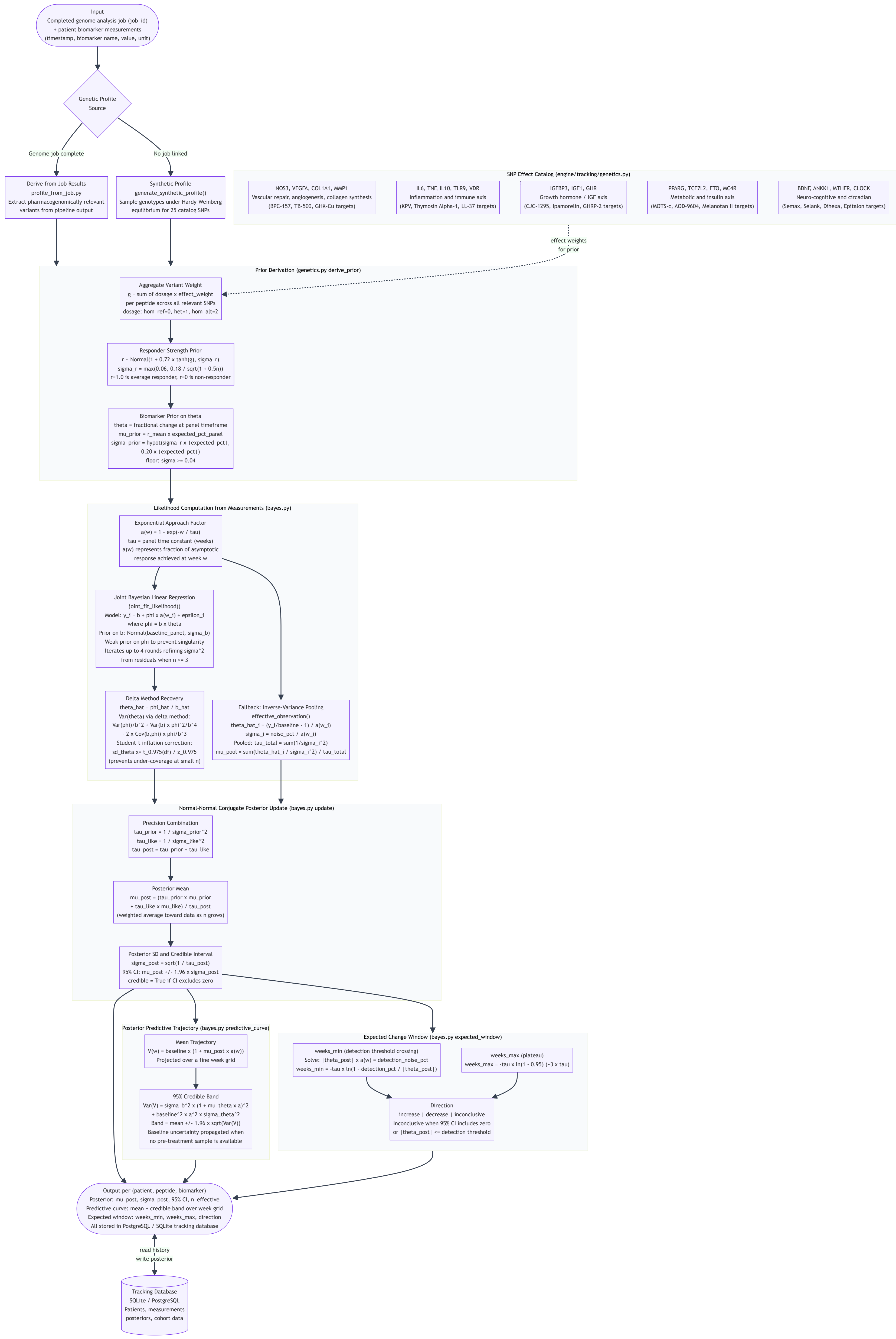


Figure 5. Bayesian biomarker inference model (engine/tracking/bayes.py and genetics.py). For each (patient, peptide, biomarker) triple the model maintains a Normal prior on theta — the expected fractional change from baseline — derived from the patient's genotype at 25 pharmacogenomically curated SNPs. As timestamped biomarker measurements arrive, a joint Bayesian linear regression simultaneously estimates the pre-treatment baseline and the effect slope (phi = baseline x theta) against the peptide's exponential approach curve a(w) = 1 - exp(-w/tau). The delta method recovers theta with appropriate uncertainty, inflated by the Student-t factor for small samples. A Normal-Normal conjugate update then combines the genetic prior with the measurement-derived likelihood, producing a closed-form posterior. The posterior is projected forward through the approach curve to generate a mean predictive trajectory with a 95% credible band, and an expected-change window (weeks_min to weeks_max) is derived from when the posterior effect first crosses the measurement noise floor. No external ML libraries are used — the inference is implemented analytically from the conjugate update equations using Python's standard math library.